// Day 24 of 30

STEP-BY-STEP GUIDE

Monolith to Microservices — Migration Guide

→ Swipe through all 6 slides

Shubham Bhati
linkedin.com/in/bhatishubham

When to Migrate

Don't migrate just because it's trendy

```
Migrate when:
  → Team > 10 devs (stepping on each other)
  → Deploy takes hours (everything coupled)
  → One bug crashes entire system
  → Different parts need different scaling

DON'T migrate when:
  → Small team (< 5 devs)
  → Low traffic
  → 'Because Netflix does it'
```

💡 Microservices solve SCALE problems, not code problems

Step 1: Identify Boundaries

Find natural seams in your monolith

```
Look for:
  → Modules that rarely change together
  → Data that belongs together
  → Teams that own different features

Example e-commerce:
  → User module → User Service
  → Order module → Order Service
  → Payment module → Payment Service
  → Inventory → Inventory Service
```

💡 Domain-Driven Design helps identify boundaries

Step 2: Strangler Fig Pattern

Don't rewrite. Migrate piece by piece.

```
Old monolith keeps running!

1. Pick ONE module (least risky)
2. Build it as new microservice
3. Route traffic: new service
4. Verify it works
5. Remove from monolith
6. Repeat for next module

→ Zero downtime migration
→ Can rollback anytime
```

💡 Strangler Fig = safest migration strategy

Shubham Bhati
linkedin.com/in/bhatishubham

Step 3: Database Per Service

This is the hardest part

```
Monolith: 1 shared database

Microservices: each owns its DB

Migration:
  1. Identify which tables belong where
  2. Duplicate data initially
  3. New service writes to own DB
  4. Sync period (both DBs)
  5. Cut over to new DB
  6. Remove old tables

→ Takes weeks. Plan carefully.
```

💡 Database split = most complex part of migration

Step 4: Communication

Services need to talk

```
Synchronous (REST/gRPC):
  → When you NEED the response now
  → GET /users/123 → immediate response

Asynchronous (Kafka/RabbitMQ):
  → When response can wait
  → Order created → event → notification

Rule: async by default,
      sync only when necessary
```

💡 80% async, 20% sync = resilient system

Step 5: Observability

You can't debug what you can't see

```
MUST have before going live:

1. Centralized logging (ELK Stack)
   → All service logs in one place

2. Distributed tracing (Zipkin)
   → Follow request across services

3. Health checks (/actuator/health)
   → Know what's up/down

4. Metrics (Prometheus + Grafana)
   → Response times, error rates
```

💡 Observability is not optional in microservices

```
done = true
```

Follow for more!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham